

Position applied for: Senior Front-End Engineer

Section 1 — Code

Q1

React.memo does a shallow compare (Object.is on each prop). It never walks into the columns array, so the “deep comparison” claim is incorrect. Memo isn’t broken, it’s doing exactly what it’s supposed to. The columns array is a new reference every render, so the shallow check fails and the rows re-render.

Wrapping columns in useMemo can help, but only if that’s the only unstable prop. If you also pass a new onClick handler, a freshly mapped rows array, or anything else that changes identity, memo still loses. Same problem if the memoised component itself is declared inside the parent, or if the rows consume changing context.

The real cause of the re-renders is simply that the parent re-renders on every keystroke. Memo only decides whether the child rows pay the cost of that. The expense comes from thousands of rows doing work because their props keep changing.

Q2

Three real problems, ranked by how bad they are for users:

1. No rollback on failure.

The catch block does nothing with the patch. RTK Query only runs invalidatesTags on a successful mutation, so if the request fails the optimistic change just sits there forever. The user sees the new status even though the server rejected it. That’s even worse than a performance issue. Happens on any network or HTTP error.

2. The optimistic update targets the wrong cache entry.

updateQueryData is called with `{}` as the filter argument. In real usage the list is almost always filtered or paginated, so the cache key on screen is different. The patch either does nothing or updates an entry the user isn't looking at. Result: no visible optimistic update most of the time.

3. Tags are too coarse.

providesTags / invalidatesTags on the whole list means every status change refetches every active getProducts query. You get a burst of extra requests whenever more than one list is mounted.

The comment "the invalidation will refetch anyway" is false. Invalidation only happens on success. On failure nothing refetches, which is exactly why the bad optimistic value sticks around.

The bug is the missing undo, not the pattern itself.

Q3

```
export function useSupplierName(supplierId: string) {  
  const { data, isLoading, isError } = useGetSupplierQuery(supplierId);  
  return { name: data?.name ?? "", isLoading, isError };  
}
```

```
export const SupplierBadge = memo(function SupplierBadge({ supplierId }: {  
  supplierId: string }) {  
  const { name, isLoading, isError } = useSupplierName(supplierId);  
  if (isLoading) return <Skeleton className="h-5 w-24" />;  
  if (isError || !name) return null;
```

```
return <span className="rounded bg-muted px-2 py-0.5  
text-xs">{name.toUpperCase()}</span>;  
});
```

What I removed and why:

- The useState + useEffect that copied data.name into local state.

That's the source of the cross-row bug. When the table reorders and a badge that previously showed supplier A gets reused for supplier B, the query for B can return cached data immediately (isLoading false). The component still has A's name in state until the effect runs. If B's data is still loading or has no name, A's name stays on B's row. One row shows another row's data.

- The useMemo around the upper-cased label.

It was memoising a value derived from the stale state copy, so the wrong label could linger for a frame. Once the state mirror is gone you can just derive it directly.

The rewrite still doesn't solve the bigger problem at 200 rows: each badge fires its own query. That needs to be fixed higher up either by including the supplier names in the products response or with a batch endpoint. Not something this component should solve.

Q4

What the user actually sees: idle → saving → idle.

The click handler sets "saving" synchronously, so that paints. After the 400 ms success, React 18 batches the two setLabel calls that run in the same microtask: first "saved", then the final line that reads the closed-over label

(which is still “idle” from the original render). The batch keeps the last value, so “saved” never appears. The same thing happens on the failure path “failed” gets overwritten by “idle”.

final value is idle. The user gets no lasting feedback that the action succeeded or failed. They might click again or assume nothing happened.

Smallest fix: delete the last `setLabel` line. The earlier `setLabel('saved')` / `setLabel('failed')` already do the right thing.

Section 2 — Design and judgement

Q5

Centralise the envelope handling in one place so the rest of the app never has to look at it.

```
export type ApiError = { code: ErrorCode; message: string; field?: string };

export async function api<T>(path: string, init?: RequestInit): Promise<T> {
  const body: ApiResponse<T> = await fetch(path, init).then(r => r.json());
  if (body.error) throw new ApiErrorThrown(body.error);
  return body.data; // TypeScript already knows this is T
}
```

```
export type ErrorSink = {
  onField(field?: string): void;
  onToast(message: string): void;
};
```

```
export function showError(err: unknown, sink: ErrorSink) {
```

```

if (!(err instanceof ApiErrorThrown)) {
  sink.onToast('Unexpected error');
  return;
}
const e = err.error;
switch (e.code) {
  case 'VALIDATION_FAILED':
    sink.onField(e.field);
    break;
  case 'RATE_LIMITED':
    sink.onToast('Too many requests — try again.');
```

break;

```

  case 'SUPPLIER_LOCKED':
  case 'STOCK_NEGATIVE':
  case 'IMPORT_IN_PROGRESS':
    sink.onToast(e.message);
    break;
  default: {
    const _exhaustive: never = e.code; // new member becomes a compile
error
    sink.onToast(e.message);      // unknown runtime code still surfaces
  }
}
}

```

Forms pass an onField that attaches the message to the right input (and falls back to a toast). Tables just toast. Background polls pass no-ops so they stay silent. None of them parse the raw envelope.

1. After the `if (body.error) throw`, TypeScript narrows the union, so `body.data` is already typed as `T`. No cast needed.
2. The `never` assignment in the default case forces a compile error when someone adds a new `ErrorCode` without handling it.
3. At runtime an unknown code still hits the default and calls `onToast`, so the message reaches the user instead of disappearing.
4. When `error.field` doesn't match any form field name, the form's `onField` simply finds nothing and falls back to a toast. That mapping lives in the form, not in the shared layer.

Q6

Reject the virtualisation proposal.

@tanstack/react-virtual only keeps the visible rows in the DOM. Everything else becomes spacers. Ctrl-P therefore prints only the ~40 rows that happen to be on screen. The warehouse team ends up with incomplete paper lists. Ctrl-F has the same problem it can't find an order that's scrolled out of view. Six seconds for 3 000 rows also suggests the real cost isn't the number of DOM nodes; it's probably heavy row components or unstable props.

Virtualisation wouldn't fix that.

Better approach: keep every filtered row in the DOM, make the rows cheap (memo + stable props), and use `content-visibility: auto` so the browser can skip work for off-screen rows. Add a print stylesheet that forces `content-visibility: visible`. Cost is some engineering time to stabilise the row props and the fact that you still have a few thousand real DOM nodes.

Q7

I'd fix the forms (b).

A four-second freeze is bad, but a form that silently throws away what the user typed is worse. People lose work, lose trust, and have to re-type (In fact i

have experienced this before). You can demo the products table with a smaller or pre-filtered set and just be honest that the full 12 k rows still need work. You can't demo your way around data loss.

To the person who wanted virtualisation: we can show a responsive version of the table for the demo.

Q8

Several pairs can't both be true:

- (2) and (3): "Select all" can include thousands of products that have never been loaded. You can't show an exact list of SKUs without fetching them all first, and even then the 500-id limit (4) means you couldn't submit them in one go.

I'd keep (2). Confirmation shows the count plus a preview of the first N SKUs and "and M more". Question for the business: is a count + sample enough, or do they really need every SKU listed?

- (5) and (4): True atomicity across an unbounded selection is impossible when the API only accepts 500 ids. You'll have to chunk, and chunks succeed or fail independently.

Keep the 500 limit (it's a hard constraint) and relax (5) to "atomic per batch, with a clear way to retry the failed ones".

- (5) and (6): If the whole operation is truly atomic there are no partial results, so a toast that reports successes and failures only makes sense once you accept partial failure.

Keep (6).

- (1) and (2) can coexist if you treat the selection as an explicit set of ids.

"Select all" just adds the current filter's matches to that set. After the user changes filters, previously selected items that no longer match can stay selected. Worth confirming with the business whether that's the desired behaviour.

Four of the six requirements can stay as written: 1, 2, 4 and 6.

Section 3 — Review and judgement under pressure

Q9

Verdict: request changes.

1. FilterBar.tsx — clear does `window.location.href = '/products'`. That's a full reload and directly breaks AC-2. Just call `onChange` with an empty suppliers array (and clear the draft).

2. `useProducts.ts` — `refetchOnMountOrArgChange: true` is unnecessary. RTK Query already treats different filter objects as different cache entries. The flag forces extra refetches and looks like it's papering over the real state problem. Remove it.

3. We can't tell from the diff whether the API actually accepts multiple suppliers or how the array is serialised. That needs checking before AC-1 can be considered done.

4. The controlled-component conversion and the `date-util` move are unrelated to the ticket. They should be separate PRs.

5. Minor: the Add button doesn't clear the draft, allows duplicates, and Enter doesn't submit.

I didn't comment on the `formatDate` move itself; it's claimed to be unchanged and out of scope.

AC-1: can't confirm from this diff alone; need to see the backend contract and a multi-supplier test.

AC-2: clearly not met.

Q10

First hour:

1. I will tell the team to stop pushing/pulling development and temporarily block force-pushes.

2. I will get the previous tip of development and the four PR branches back from anyone who still has a recent clone (reflog or fetch of the PR refs).
3. I will also restore the old SHA to development, recreate the branches, reopen the PRs.
4. Then confirm the history looks right and ask the developer to turn off whatever auto-sync did this.
5. I will immediately leave the branch protection "on" permanently and agree a clear force-push policy. I will go ahead. Tell the two blocked developers immediately: stop touching development, your work is safe, send me your reflog if you have one. Once restored, I will tell them the new SHA and that they can pull again.

Business owner: I will say nothing until it's fixed (that is production is fine). If it drags on more than a couple of hours, I will give a short factual update.

Afterwards one sentence is enough: branch was accidentally rewritten, restored in about an hour, no production impact, protections added.

Then lastly, Permanent change: force-push to development is blocked. That needs agreement from the developer and the rest of the team.

Q11

To the developer:

Hello, I want to talk about PR size, especially last week's merge. 900+ lines and no tests meant nobody could really review it properly and a bug got through. I know splitting feels slower at the moment, but the evidence is the other way: smaller PRs get reviewed faster, catch more issues, and cause less rework later. Going forward let's aim for slices that are reviewable in under half an hour and that include tests for the new behaviour..

To the business owner:

We're changing how we break up work so changes stay smaller and get proper review before they ship. Big untested PRs look faster in the short term

but they let bugs through and slow the next release down. Last week's deadline pressure is a good example. Feature dates stay the same; we're just reducing the risk that comes with very large changes.

Section 4 — Experience

Q12

I once shipped a bug that made the order-history page show “no orders” for every customer. The API returned camelCase fields (orderDate, totalAmount). I had hand-written the TypeScript type with different names, so every field came through as undefined. The row filter treated them as empty and the table rendered nothing.

What changed afterwards wasn't just the code. I stopped hand-writing response types; we generate them from the OpenAPI spec. Every data screen now has a simple contract test that feeds a fixture response and asserts rows appear. Field names get agreed with the backend person before the UI work starts.

Q13

1. An internal business-management tool. Used daily by the founder and a small ops team.

The api was designed by a colleague. We agreed the response shapes in a shared TypeScript type file and reviewed sample payloads before coding. The hardest part was keeping the summary cards and chart consistent on refresh without layout shift.

2. Transaction dashboard for a cross-border payments platform.

Let business users initiate and track international payments, view multi-currency balances, and monitor settlement status across corridors. Used by finance and operations teams at client companies.

I contributed to the backend with the rest of the backend team. We locked the contract via shared OpenAPI specs and a short set of example payloads in the ticket.

The hardest part was handling real-time status updates and FX rate changes without over-fetching or showing stale balances.